CE-302 SOFTWARE ENGINEERING LABORATORY · TE CE D1 · SEM V · AY 2026-27

Two key modules of the NGO Volunteer & Donation Management System

Donation Processing **(SRS §3.3)** and Fund Utilization Tracking & Reporting **(SRS §3.6)**: requirements, design, implementation and verification.

Module A · money in

A donor gives to an Active campaign. The payment is processed, the campaign total updates, and a numbered receipt is issued.

Module B · money out

Staff record how the money was spent. Spending can never exceed what was raised, and every entry leaves an audit trail.

Together they implement the NGO's promise of **accountability**: every rupee is received, receipted, spent and reported on the record.

PROJECT	NGO Volunteer and Donation Management System, Release 1.0 (SRS dated 30/07/2026)
SRS AUTHORS	Shubh Srivastava (2024300249) · Akshit Sharma (2024300234)
INSTITUTE	Sardar Patel Institute of Technology
STACK	React + TypeScript · FastAPI (Python 3.11) · SQLAlchemy · SQLite / PostgreSQL

1 · CONTEXT

The project on one page

A web platform that lets an NGO run donation campaigns, accept donations, manage volunteers and events, and show how collected funds were used. It replaces spreadsheets and paper registers, which are error-prone and hard to audit (SRS §1.4).

USERS	**Donor** (registered or guest) · **Volunteer** (needs admin approval) · **Admin** (NGO staff)
MODULES (SRS §3)	3.1 Volunteers · 3.2 Campaigns · **3.3 Donation Processing** · 3.4 Donors · 3.5 Events & Tasks · **3.6 Fund Utilization & Reporting**
DELIVERED	56 API paths (64 operations), 56 automated backend tests, a responsive web app with Donor, Volunteer and Admin areas

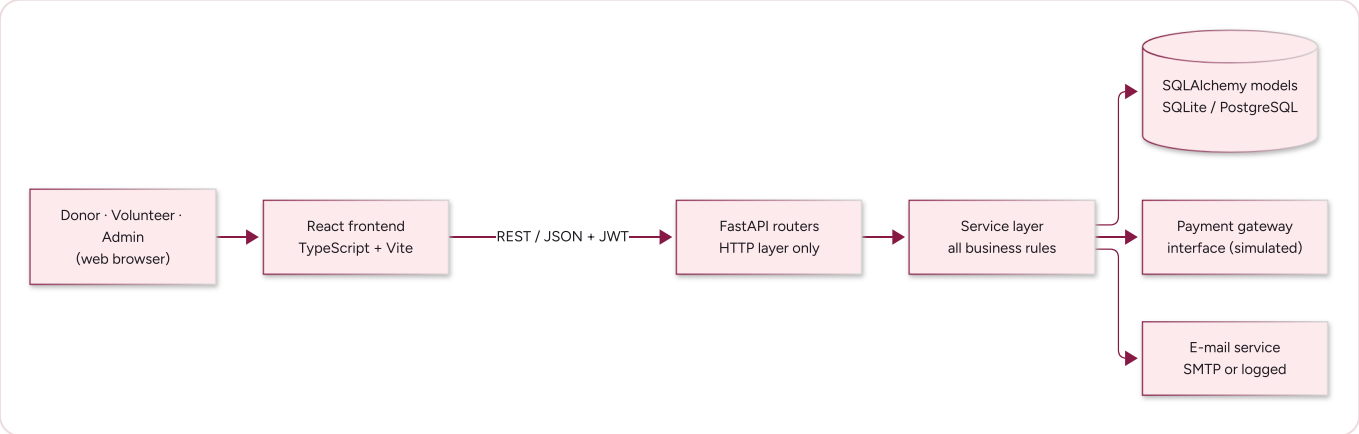


Figure 1 · Layered architecture. Routers only handle HTTP, all rules live in the service layer, and the two external dependencies (payment and e-mail) sit behind interfaces.

Why these two modules?

We chose the two modules that **follow the rupee**. They carry the most business rules and the highest risk (money, concurrency, security), they depend on each other, and together they are the reason the system exists. The volunteer/event modules follow the same layered pattern, so everything explained here applies to them too.

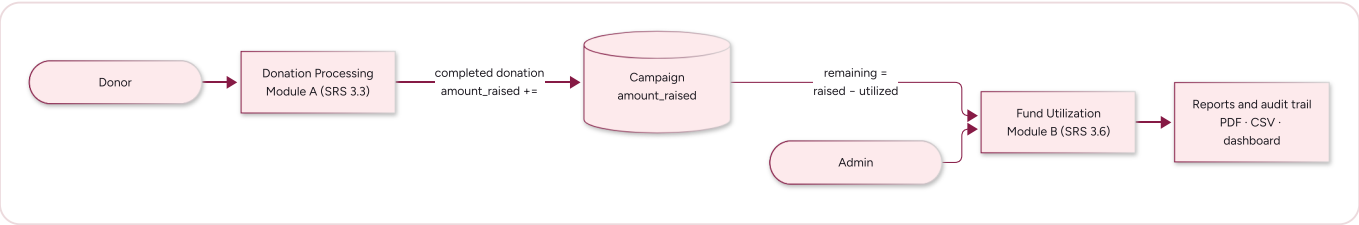


Figure 2 · How the two modules connect: a completed donation raises the campaign total, and fund utilization is allowed only up to that total.

Donation Processing *(SRS §3.3)*

A registered or guest donor makes a one-time donation to an Active campaign. The system validates it, processes a (simulated) payment, records the result, updates the campaign total and issues a receipt.

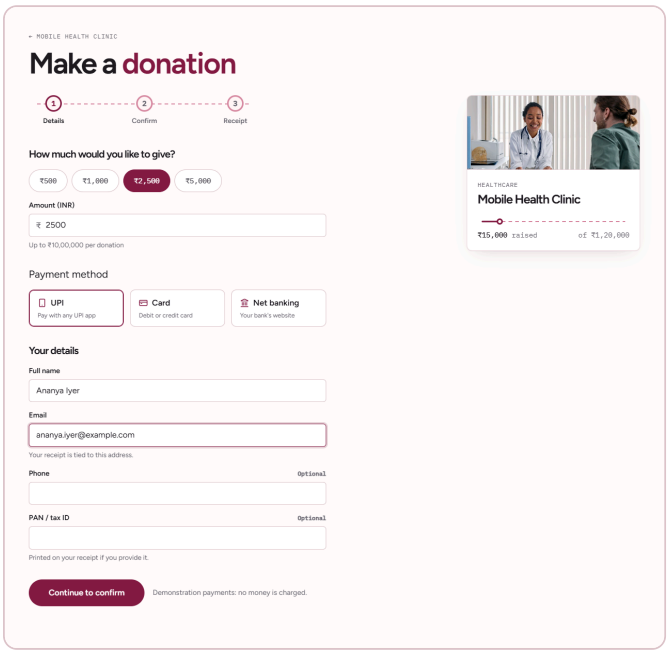
SAY THIS IN 30 SECONDS

"Donating is a two-call process. *Initiate* creates a pending donation and shows a confirmation summary. *Confirm* runs the payment inside a lock, so a donation can only complete once. Success updates the campaign total and issues a receipt. Failure changes nothing, and the donor can retry."

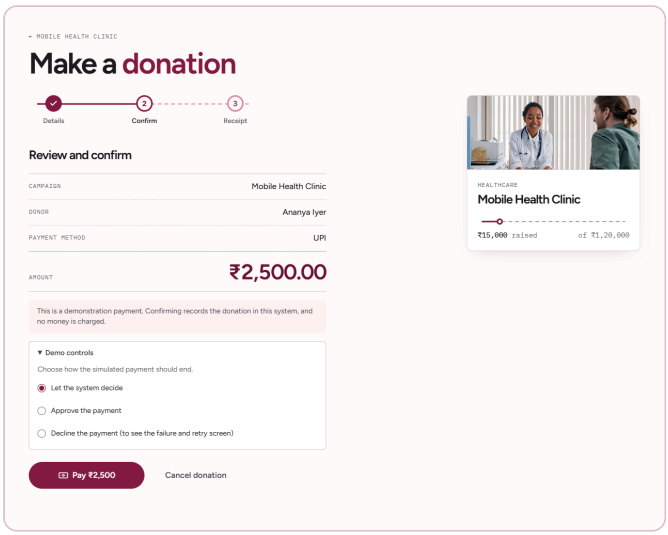
2.1 Requirements covered and where they are implemented

SRS ID	WHAT THE SRS REQUIRES	HOW WE MEET IT	WHERE
Donation.Initiate	Pick an Active campaign and an amount	`POST /donations` creates a **pending** donation	`initiate_donation`
Donation.Amount.Validate	Reject zero, negative or over-limit amounts	Schema: > 0, ≤ 2 decimals. Service: ≤ `MAX_SINGLE_DONATION` (config, resolves TBD-1)	schemas, `donations.py`
Donation.Guest	Donate without an account (name, e-mail, optional PAN)	`donor` object in the body; donor record created or reused	`_resolve_donor`
Donation.Payment.Select	Card, UPI or net banking	`payment_method` enum, validated	`PaymentMethod`
Donation.Confirm	Show a summary before processing	Response carries a `confirmation` block; payment happens only on a separate `/confirm` call	`DonationInitiated`
Donation.Process.Success	Record Completed, update the total, issue receipt	status = completed, `amount_raised += amount`, receipt number	`confirm_donation`
Donation.Process.Failure	Record Failed and do **not** change the total	status = failed; campaign untouched; retry creates a new pending donation	`confirm_donation`, `retry_donation`
Donation.Receipt.Generate	Receipt with donor, amount, campaign, date, unique number	JSON + PDF (ReportLab); number `RCP-YYYYMMDD-nnnnnn`	`receipts.py`
Donation.Receipt.Email	E-mail the receipt where e-mail is configured	Background task after the response; logged as *skipped* when no SMTP (DE-1)	`notifications.py`
Donation.History.View	Registered donor sees own history	`GET /donors/me/donations`	`routers/donors.py`
Campaign.Progress.Update, Close.Auto	Total updates immediately; campaign closes at goal	Inside the same transaction as the donation	`sync_campaign`

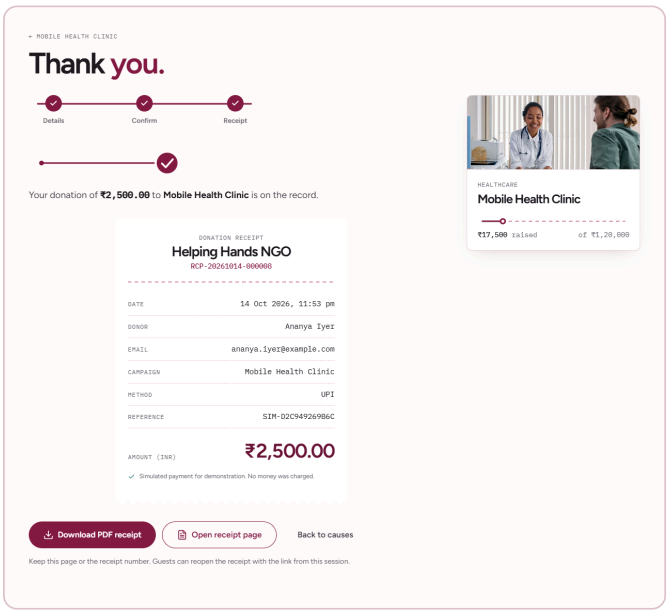
2.2 The user journey (three screens, within the SRS limit of four steps)



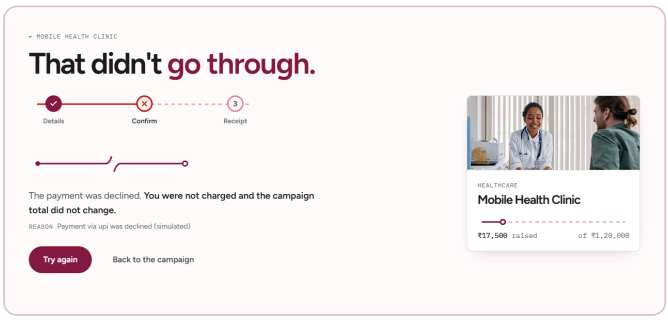
Step 1 · Details: amount, payment method and donor details, with inline validation



Step 2 · Confirm: a summary is shown before any payment is taken



Step 3 · Receipt: numbered receipt on screen and as a PDF



Failure path · payment declined: the campaign total did not change. "Try again" creates a new pending donation.

2.3 Design

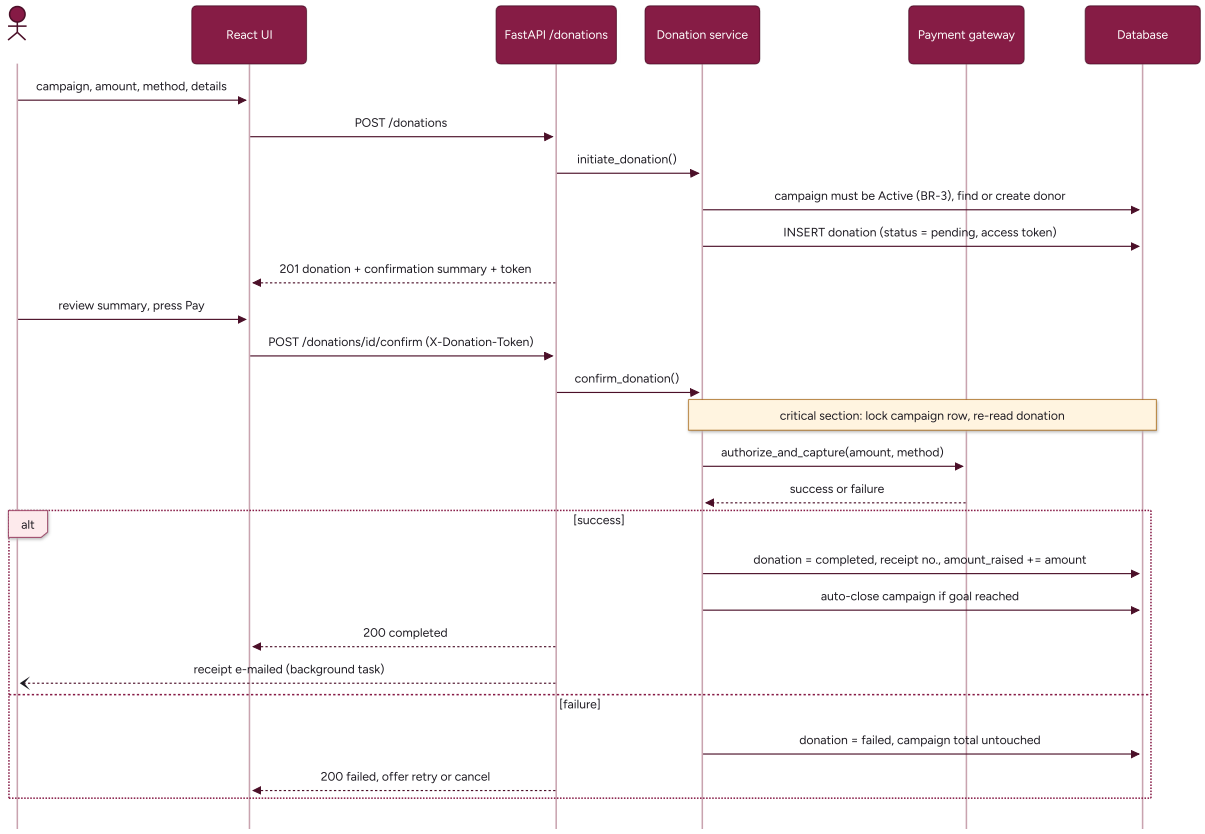


Figure A1 · Sequence diagram of a donation (guest flow). The critical section around the payment is what prevents double counting.

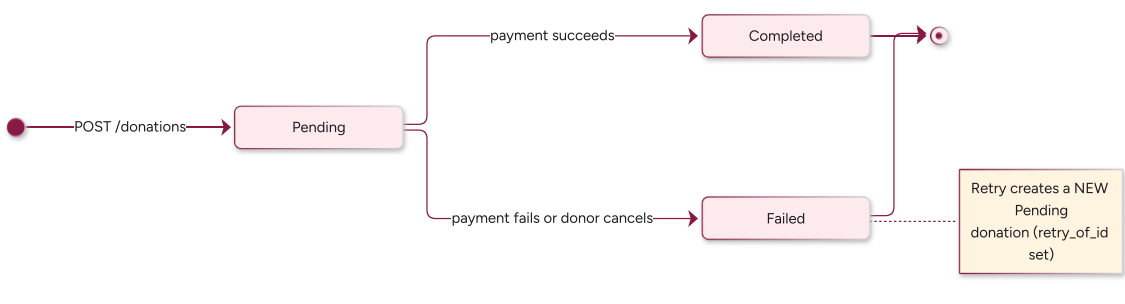


Figure A2 · Donation status (BR-2, SRS Appendix B Figure 4)

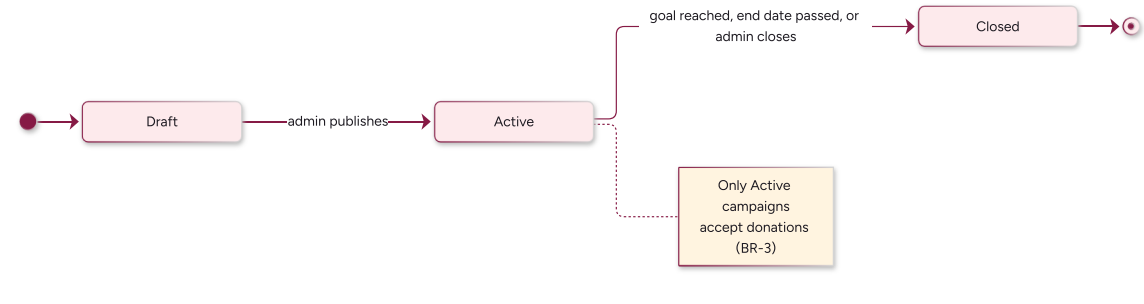


Figure A3 · Campaign status (BR-1, SRS Appendix B Figure 3)

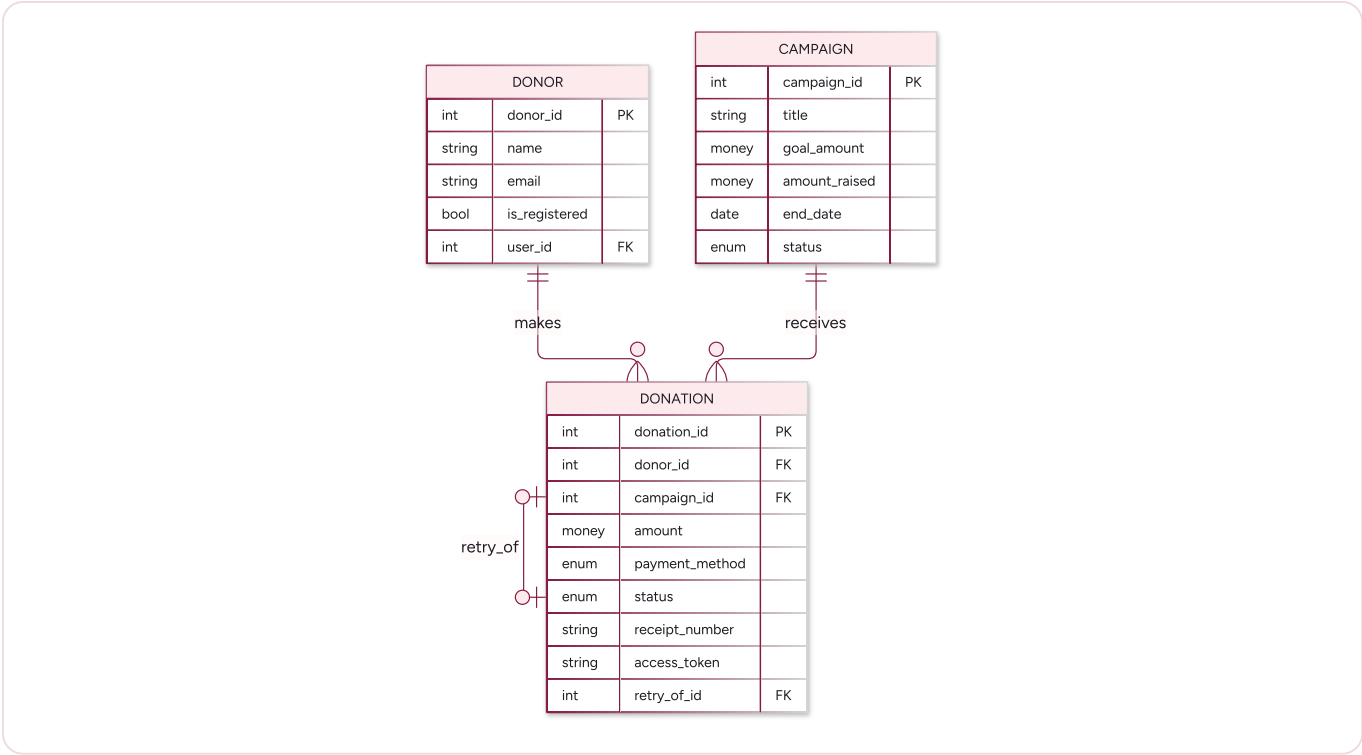


Figure A4 · Data model for the module (Appendix A of the SRS). Money columns hold integer paise.

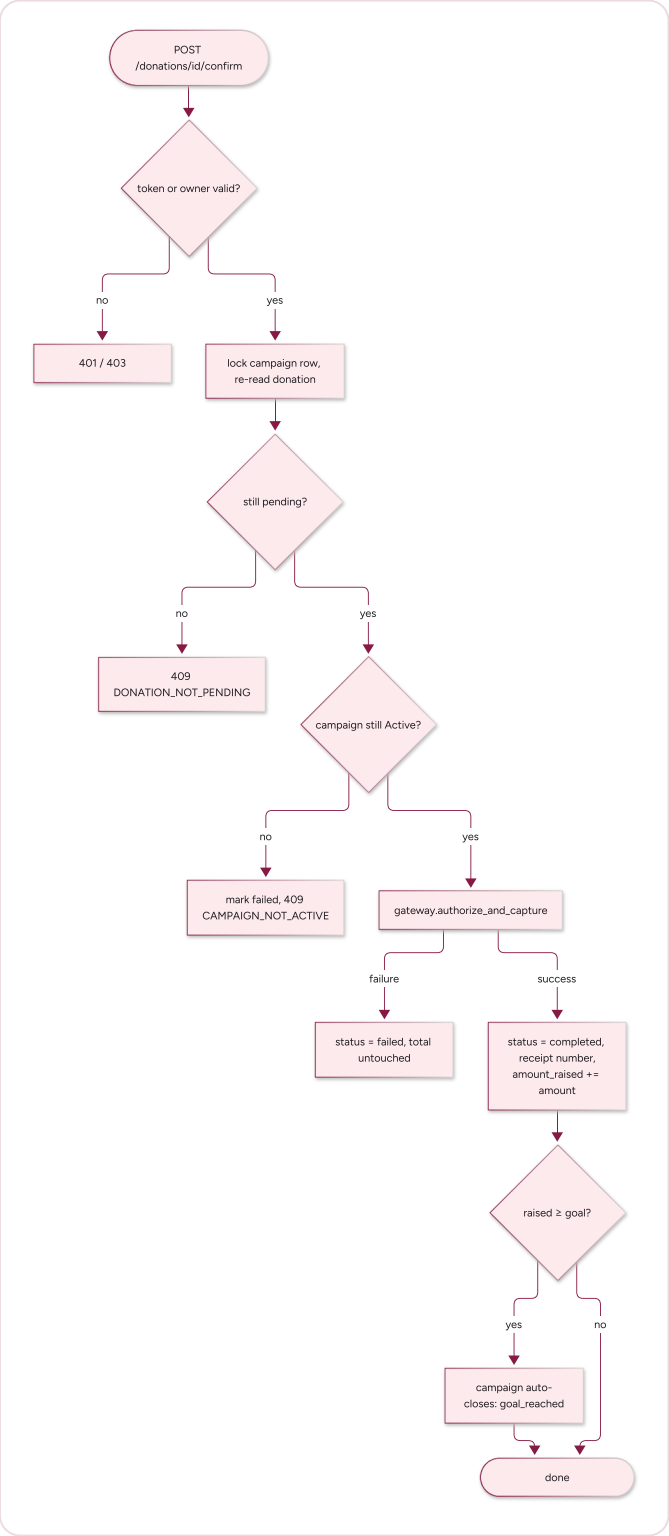


Figure A5 · Activity diagram of the confirm step: every early exit leaves the campaign total unchanged.

How to read Figure A5

1. **Who is calling?** A guest must send the donation token, a donor their login. Anyone else gets 401 or 403 (SE-5).
2. **Lock, then re-read.** The campaign row is locked and the donation is read again, so a double-click cannot be processed twice.
3. **Still pending?** If not, the answer is 409 `DONATION_NOT_PENDING` and nothing changes.
4. **Still Active?** The campaign may have closed since step 1. If so, the donation is marked failed (BR-3).
5. **Call the gateway.** A failure marks the donation failed and leaves the campaign total untouched.
6. **Success.** The donation becomes completed, gets its receipt number, and `amount_raised` grows by the amount. If raised reaches the goal, the campaign closes itself.
7. **One commit** at the end makes all of this atomic.

Why it matters. This one function is what makes the receipt, the campaign total and the donation status always agree.

2.4 Rules and non-functional requirements, and what enforces them

RULE	STATEMENT	ENFORCED BY
BR-2	Donation status is Pending, Completed or Failed	DB CHECK constraint + enum
BR-3	Only Active campaigns accept donations	checked at initiate *and again at confirm*
CO-3	Money exact to 2 decimals	stored as integer paise (`Money` column type), shown as `"2500.00"` strings

RULE	STATEMENT	ENFORCED BY
CO-4 / SI-2	No real payment; replaceable later	`PaymentGateway` interface + `SimulatedGateway`
SE-5	A donor sees only their own donations	owner check or secret token; others get 403
PE-3	Confirm within 5 seconds	measured 5 to 20 ms locally
Usability-1	First-time donor finishes in ≤ 4 steps	3 screens: details → confirm → receipt

2.5 Key code (excerpts from the repository, not retyped)

services/donations.py

```python
def confirm_donation(db: Session, donation: Donation, forced_outcome: str | None) -> Donation:
    with critical_section:
        campaign = get_campaign(db, donation.campaign_id, lock=True)
        db.refresh(donation)  # re-read under the lock: guards against a double-submitted confirm
        if donation.status != DonationStatus.PENDING:
            raise conflict("DONATION_NOT_PENDING", f"This donation is already {donation.status.value}")

        if sync_campaign(campaign) or campaign.status != CampaignStatus.ACTIVE:
            donation.status = DonationStatus.FAILED
            donation.failure_reason = "Campaign is no longer accepting donations"
            db.commit()
            raise conflict("CAMPAIGN_NOT_ACTIVE", "This campaign is no longer accepting donations")

        result = get_gateway().authorize_and_capture(donation.amount, donation.payment_method, forced_outcome)
        donation.payment_reference = result.reference
        if result.success:
            now = utcnow()
            donation.status = DonationStatus.COMPLETED
            donation.completed_at = now
            donation.receipt_number = make_receipt_number(donation, now)
            campaign.amount_raised = campaign.amount_raised + donation.amount  # Campaign.Progress.Update
            sync_campaign(campaign)  # goal reached -> auto-close
        else:
            donation.status = DonationStatus.FAILED  # Donation.Process.Failure: total untouched
            donation.failure_reason = result.message
        db.commit()
    return donation
```

Read it top to bottom: **lock** the campaign, **re-read** the donation, stop if it is not pending, stop if the campaign is closed, **call the gateway**, then either complete (total + receipt + auto-close) or fail (nothing else changes), and commit once.

services/payment.py

```python
class SimulatedGateway:
    def authorize_and_capture(
        self, amount: Decimal, method: PaymentMethod, forced_outcome: str | None = None
    ) -> PaymentResult:
        if forced_outcome and settings.payment_allow_forced_outcome:
            success = forced_outcome == "success"
        else:
            success = random.random() >= settings.payment_failure_rate
        reference = f"SIM-{uuid.uuid4().hex[:12].upper()}"
        if success:
            return PaymentResult(True, reference)
        return PaymentResult(False, reference, f"Payment via {method.value} was declined (simulated)")
```

2.6 Design decisions worth explaining

- **Two calls, not one.** Splitting *initiate* from *confirm* gives the donor a confirmation screen (Donation.Confirm) and makes the payment step safely repeatable.
- **Gateway behind an interface (Strategy/Adapter idea).** Swapping in Razorpay or Stripe later changes one class, not the donation rules (SI-2.2).
- **Integer paise.** Floating point cannot store 0.10 exactly (CO-3), so we store integers and send strings like `"2500.00"`.
- **Guest access token.** A random 256-bit secret, returned once and compared in constant time, lets a guest confirm and fetch a receipt without an account.
- **Defence in depth.** The UI validates and disables buttons to help users, but every rule is enforced again on the server.

- **Failure is a normal outcome.** A declined payment returns HTTP 200 with `status: failed`; retry creates a *new* pending donation, so the history stays honest.

2.7 Verification of this module

- **Unit/integration tests:** `test_guest_donation_full_flow_updates_total_and_issues_receipt`, `test_failed_payment_does_not_change_total_and_can_be_retried`, `test_double_confirm_is_rejected_and_counts_once`, `test_amount_validation`, `test_access_control_on_flow_endpoints`, `test_registered_donor_flow_and_history_privacy`, `test_auto_close_when_goal_reached`, `test_money_is_exact_to_two_decimals`.
- **Live concurrency test:** 12 simultaneous confirms of one pending donation produced **one** success and eleven 409 responses; the campaign total rose exactly once.
- **Performance (PE-3):** initiate about 6 ms and confirm 5 to 20 ms on a laptop, against the 5 second requirement.
- **Browser test:** success with PDF download, decline then retry (total unchanged by the decline), refresh on the confirm step, cancel, and a closed campaign refusing donations.

Fund Utilization Tracking & Reporting *(SRS §3.6)*

An Admin records how the money raised by a campaign was spent. The system guarantees that total spending never exceeds the amount raised, keeps an audit trail of who recorded what and when, and produces a per-campaign report.

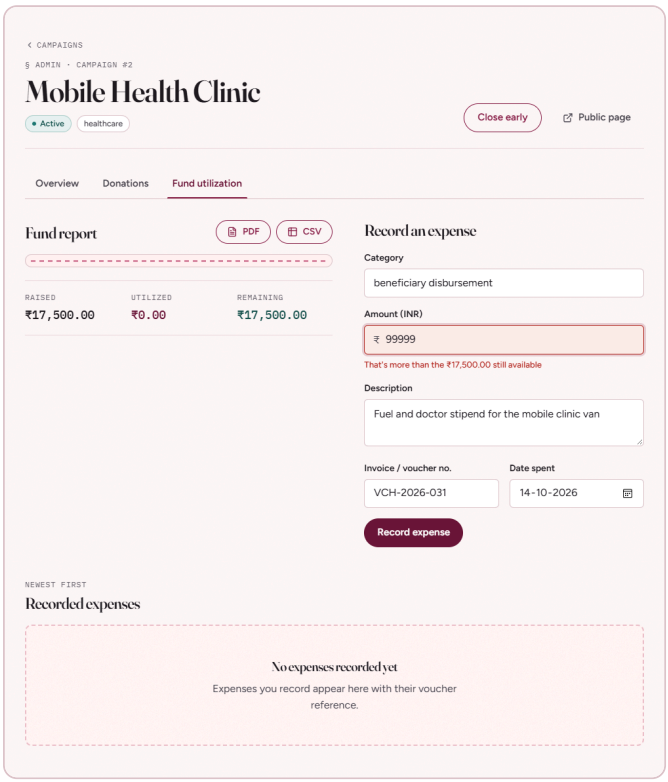
SAY THIS IN 30 SECONDS

"Each expense carries a category, amount, description, date and an invoice or voucher number. Before saving, the server locks the campaign and checks that the amount fits in what is still unspent. If not, it refuses. Every entry stores the admin and a timestamp, and the same data feeds the report, the PDF/CSV export and the dashboard."

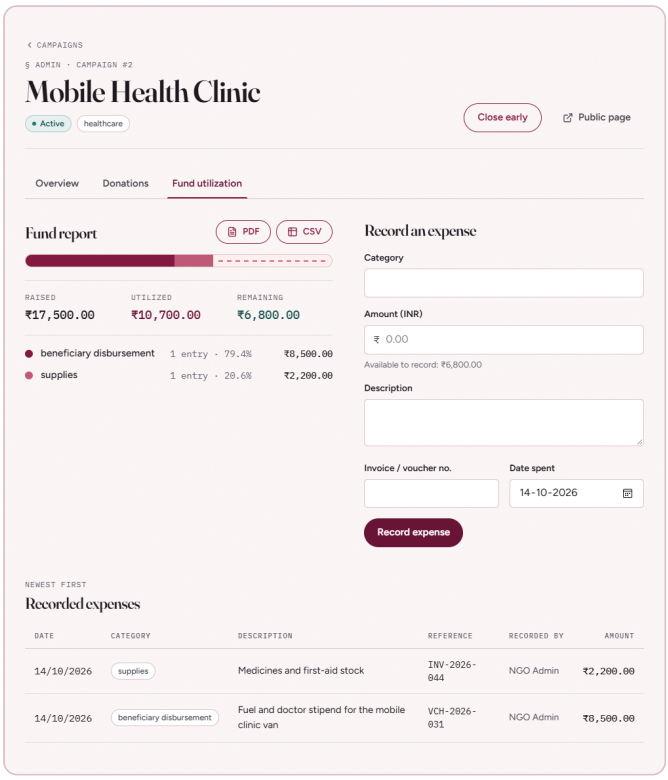
3.1 Requirements covered and where they are implemented

SRS ID	WHAT THE SRS REQUIRES	HOW WE MEET IT	WHERE
Fund.Utilization.Record	Record category, amount, description, date, reference against a campaign	`POST /admin/campaigns/{id}/utilizations`	`record_utilization`
Fund.Utilization.Validate, BR-4	Reject an entry if total utilized would exceed total raised	Remaining is computed under a lock; over-limit gives **400 UTILIZATION_EXCEEDS_RAISED**	`funds.py`
Fund.Report.Campaign	Raised, utilized, remaining and category breakdown	`GET …/fund-report`	`build_report`
Fund.Report.Export	Export as PDF or CSV	`GET …/fund-report/export?format=pdf|csv`	`report_pdf` / `report_csv`
Fund.Report.Dashboard	Totals across all campaigns on the Admin dashboard	`GET /admin/dashboard`	`admin_dashboard`
BR-5, SE-4	Only an Admin may record fund use	one admin guard on the whole `/admin` router	`routers/admin/__init__`
SE-6	Log every entry with admin id and timestamp	`recorded_by`, `recorded_date` + audit-log row in the same transaction	`log_action`
CO-3	Exact money	same integer-paise column type as donations	`Money`

3.2 The Admin experience



Recording an expense above the remaining balance is rejected inline, showing the real remaining amount (BR-4).



After valid entries: the spool shows spending by category, with report totals, PDF/CSV export and the ledger of entries.

3.3 Design

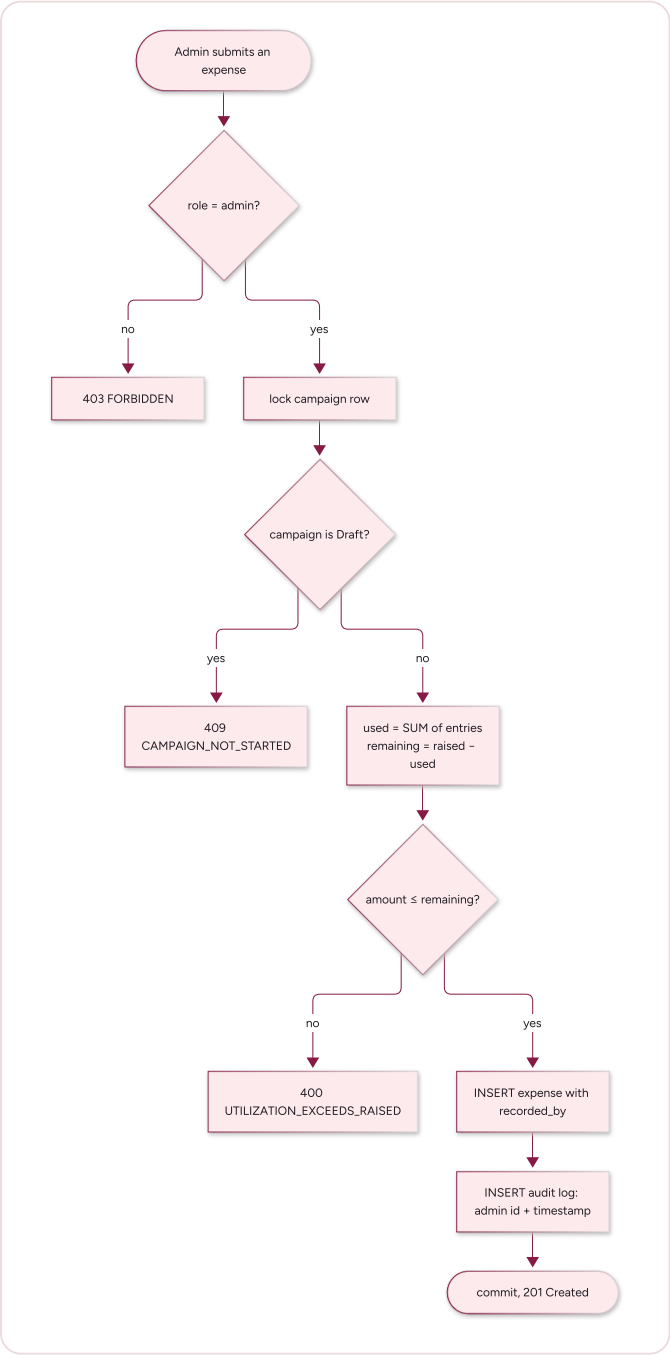


Figure B1 · Activity diagram of "record an expense". Three checks guard the write: admin role, campaign not Draft, amount fits the remaining balance.

How to read Figure B1

1. **Admin only.** One guard on the whole `/admin` router refuses everyone else with 403 (BR-5, SE-4).
2. **Lock the campaign.** Two admins entering expenses at the same moment cannot overspend together.
3. **Not a Draft.** A Draft campaign cannot have raised anything, so it is refused with 409 (`CAMPAIGN_NOT_STARTED`).
4. **Work out what is left.** remaining = `amount_raised` − the sum of all earlier entries.
5. **Does it fit?** If the amount is larger than what remains, the answer is 400 `UTILIZATION_EXCEEDS_RAISED`, and the message states the maximum allowed (BR-4).
6. **Save with proof.** The expense is stored with `recorded_by`, and the audit row is added in the *same transaction* (SE-6).

Example from the demo. Raised ₹17,500.00. Entering ₹99,999 is refused with "more than the ₹17,500.00 still available". ₹8,500 and ₹2,200 are accepted, leaving ₹6,800.00.

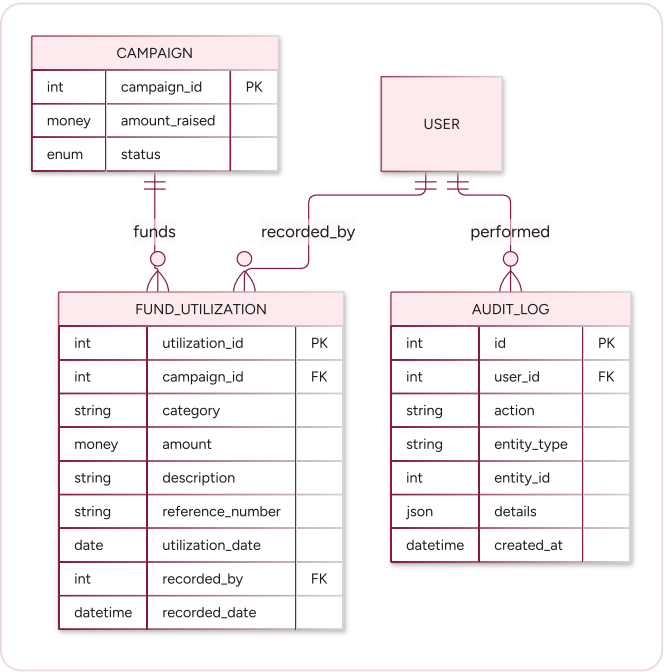


Figure B2 · Data model: each expense belongs to a campaign and records the admin who entered it. The audit log stores the same action independently.

How the report is calculated

FIGURE	DEFINITION
Raised	campaign `amount_raised` (completed donations only)
Utilized	Σ amount of all expense entries
Remaining	raised − utilized
Breakdown	entries grouped by category, with each share of utilized
Dashboard	the same three figures summed across all campaigns

All amounts are integer paise in the database and exact 2-decimal strings in the API, so the three figures always add up.

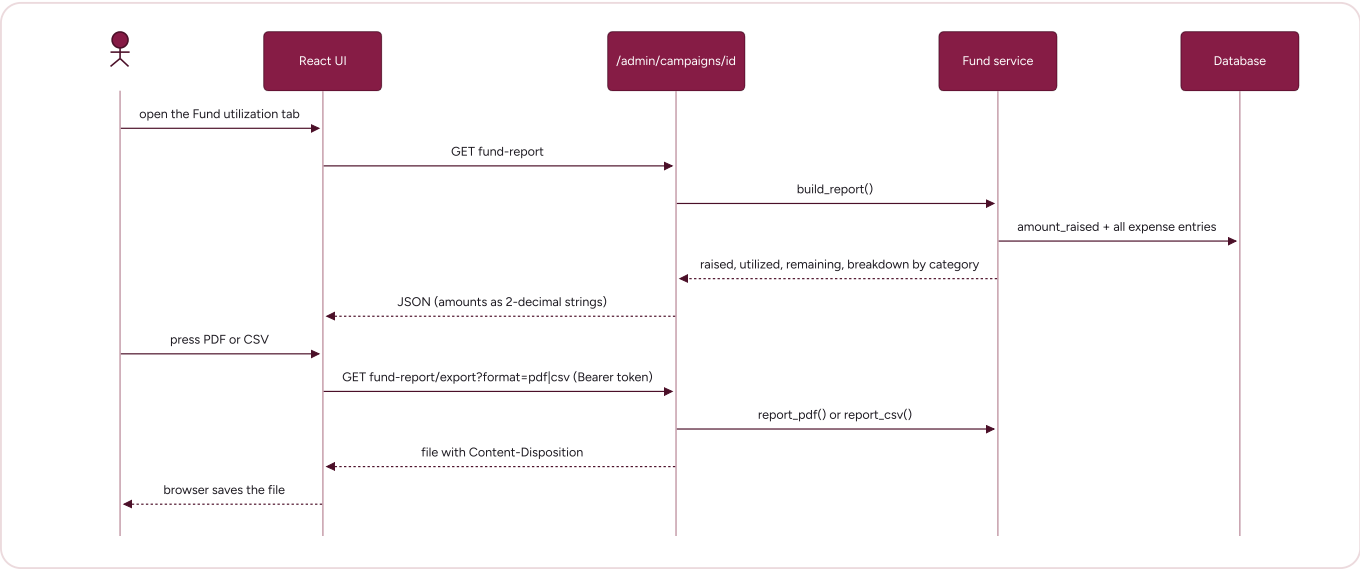


Figure B3 · Sequence diagram of opening the report and exporting it. Export is fetched with the Authorization header and saved as a file.

3.4 Key code (excerpt from the repository)

services/funds.py

```python
def record_utilization(db: Session, admin: User, campaign_id: int, data: UtilizationCreate) -> FundUtilization:
    with critical_section:
        campaign = get_campaign(db, campaign_id, lock=True)
        if campaign.status == CampaignStatus.DRAFT:
            raise conflict("CAMPAIGN_NOT_STARTED", "Funds can only be recorded against an Active or Closed campaign")

        used = total_utilized(db, campaign_id)
        remaining = campaign.amount_raised - used
        if data.amount > remaining:  # Fund.Utilization.Validate / BR-4
            raise AppError(
                400,
                "UTILIZATION_EXCEEDS_RAISED",
                f"Amount exceeds the remaining balance of {remaining:.2f} {settings.currency} "
                f"(raised {campaign.amount_raised:.2f}, already utilized {used:.2f})",
                errors=[{"field": "amount", "message": f"Maximum allowed is {remaining:.2f}"}],
            )

        entry = FundUtilization(
            campaign_id=campaign_id,
            category=data.category,
            amount=data.amount,
            description=data.description,
            reference_number=data.reference_number,
            utilization_date=data.utilization_date or local_today(),
            recorded_by=admin.id,
        )
        db.add(entry)
        db.flush()
        log_action(  # SE-6: audit trail with admin id + timestamp
            db, admin, "fund.utilization", "fund_utilization", entry.utilization_id,
            {
                "campaign_id": campaign_id,
                "amount": f"{data.amount:.2f}",
                "category": data.category,
                "reference_number": data.reference_number,
            },
        )
        db.commit()
    return entry
```

The three guards appear in the order of Figure B1. The audit row is added in the **same transaction** as the expense, so an expense can never exist without its audit trail (SE-6).

3.5 Design decisions worth explaining

- **The invariant is checked where it can break.** "Utilized ≤ raised" is verified at write time under a lock, not only when reporting.
- **Server is the authority.** The form pre-checks and shows the remaining amount to be helpful, but a request that skips the UI is still refused (`400 UTILIZATION_EXCEEDS_RAISED`).
- **Immutable by design.** There is no edit or delete for an entry. Corrections are new entries, which keeps the audit trail trustworthy.
- **Closed campaigns still accept entries.** Spending usually happens after fundraising ends; only *Draft* campaigns are refused (nothing could have been raised).
- **Role enforcement in one place.** One guard protects every `/admin` route, so a new endpoint cannot accidentally be left open (BR-5, SE-4).

3.6 Verification of this module

- **Tests:** `test_utilization_cannot_exceed_amount_raised` (exact remainder allowed, one paisa more refused), `test_utilization_requires_started_campaign_and_valid_input`, `test_fund_report_and_dashboard` (checks every figure), `test_fund_report_export_pdf_and_csv`, `test_fund_endpoints_are_admin_only`, `test_audit_trail_records_admin_and_timestamp`.
- **Browser test:** an over-limit amount shows the inline error, a valid expense appears in the ledger and the backend totals, the CSV contains the voucher number, the PDF is a valid file, and the audit trail lists the admin.

The invariant that ties the modules together

$$\Sigma \text{ expenses recorded for a campaign} \leq \text{amount raised} = \Sigma \text{ completed donations}$$

Raised grows only through **Module A** (under a lock, completed donations only). Module B checks against it, under a lock, before every write.

This is why the two modules are explained together: Module A makes the left-hand side of the report trustworthy (failed or cancelled attempts never count), and Module B makes the right-hand side trustworthy (spending cannot be invented). The Admin dashboard then shows both sides at a glance.

Evidence summary

EVIDENCE	WHAT IT SHOWS	RESULT
Backend automated tests (pytest)	56 tests over auth, campaigns, donations, events, funds. Module A: 9 donation-flow tests plus goal auto-close and exact-money. Module B: 6 tests (expense cap, draft rule, report maths, exports, admin-only, audit trail)	**56 / 56 pass**
Concurrency test (live server)	12 simultaneous confirms of one donation	**1 × 200, 11 × 409**, total counted once
Browser end-to-end, donations	success + PDF, decline then retry, refresh restores confirm step, cancel, closed campaign blocked	**13 / 13 pass**
Browser end-to-end, admin and funds	create → publish → edit lock → cap error → record → PDF/CSV export → audit trail → close	**19 / 19 pass**
All 47 routes load	no console errors on any public, donor, volunteer or admin page	**pass**
Accessibility (axe-core)	WCAG 2.x A/AA rules on 17 pages	**0 violations**

Software-engineering concepts demonstrated

- **Requirements traceability**: every SRS ID above maps to code and a test.
- **Layered architecture / separation of concerns**: UI, routers, services, models.
- **State machines**: donation and campaign status are explicit and validated.
- **Design by contract (invariants)**: BR-3 and BR-4 are enforced, not assumed.
- **Abstraction of external systems**: payment gateway and e-mail behind interfaces.
- **Testing at several levels**: unit/integration, concurrency, browser end-to-end, accessibility.

API surface of the two modules

METHOD	PATH	WHO	PURPOSE
POST	/donations	anyone	start a donation (pending)
POST	/donations/{id}/confirm	token / owner	run the payment → completed or failed
POST	/donations/{id}/cancel	token / owner	abandon a pending donation
POST	/donations/{id}/retry	token / owner	new pending donation after a failure
GET	/donations/{id}/receipt[.pdf]	token / owner / admin	receipt as JSON or PDF
GET	/donors/me/donations	donor	own history

METHOD	PATH	WHO	PURPOSE
POST	/admin/campaigns/{id}/utilizations	admin	record an expense
GET	/admin/campaigns/{id}/utilizations	admin	list expenses
GET	/admin/campaigns/{id}/fund-report	admin	raised / utilized / remaining + breakdown
GET	/admin/campaigns/{id}/fund-report/export	admin	PDF or CSV download
GET	/admin/dashboard	admin	totals across campaigns
GET	/admin/audit-log	admin	who did what and when

5 · BE READY FOR QUESTIONS

Likely questions, with short answers

1. Why is payment "simulated", and how would a real gateway be added?

The SRS fixes this for Release 1.0 (CO-4). The donation rules talk to a `PaymentGateway` interface and never to a concrete class. A Razorpay or Stripe class only has to implement `authorize_and_capture` and be returned by `get_gateway()`. The donation rules and the API stay unchanged, which is exactly SI-2.2.

2. What stops one donation being counted twice if the donor double-clicks Pay?

Confirming is a critical section: we lock the campaign row, then **re-read** the donation and require status = pending. The second request sees completed and gets 409. We proved it live: 12 simultaneous confirms gave one success and eleven 409s, and the campaign total rose once. The button is also disabled while loading.

3. Why store money as integers?

CO-3. Floating point cannot represent 0.10 exactly, and 0.1 × 3 is 0.30000000000000004. We store paise as integers and expose two-decimal strings, so sums are exact on SQLite and PostgreSQL. A test adds ₹0.10 three times and expects exactly 0.30.

4. How do you guarantee utilized never exceeds raised (BR-4)?

Amount raised only grows through completed donations, which happen under the same lock. When an expense arrives we lock the campaign, compute remaining = raised − Σ(entries) and reject anything larger. The UI shows the remaining amount and pre-checks, but the server is the authority, so the rule holds even if someone bypasses the UI.

5. How can a guest confirm a donation securely without logging in?

Creating a donation returns a random 256-bit access token, shown once. It is sent as `X-Donation-Token` and compared in constant time. A logged-in donor can use their JWT instead; admins can read but not act. Anyone else gets 403 (SE-5).

6. What if the campaign closes between "initiate" and "confirm"?

BR-3 is checked at both points. At confirm we lock, re-evaluate auto-close, and if the campaign is no longer Active we mark the donation failed and return 409 CAMPAIGN_NOT_ACTIVE. Nothing is charged and no total changes.

7. Why a service layer?

Routers only translate HTTP. Rules such as BR-3 and BR-4 live in one place, can be unit-tested without HTTP, and are reused by the seed script and tests. This separation of concerns kept each module small (donations service 167 lines, funds service 247).

8. What are the known limitations?

Payments are simulated; there is no Alembic migration tool; guests can only reopen a receipt while their browser session holds the token (the API has no lookup by receipt number); fund reports are admin-only; testing was on SQLite, and the PostgreSQL path is configured but not exercised.

A five-minute demo script

Start both servers (backend on port 8000, frontend on 5173) with the seeded demo data. Demo accounts: admin `admin@ngo.org` / `Admin@12345`.

1. **Module A, the happy path.** Open *Causes*, pick a campaign, press *Donate*. Enter an amount and guest details. Show the confirm screen, press *Pay*, and show the receipt and the PDF download. Reopen the campaign: the raised amount increased.
2. **Module A, failure.** Donate again, open *Demo controls*, choose *Decline*. Show that the total did **not** change, then press *Try again* and approve.
3. **Module A, rules.** Try a closed campaign (blocked), and an invalid amount such as 0 (inline error).
4. **Module B.** Sign in as admin, open a campaign, choose *Fund utilization*. Enter an amount larger than the remaining balance to show the refusal.
5. **Module B, recording.** Enter a valid expense with a voucher number. Show the spool, the ledger and the changed remaining balance.
6. **Evidence.** Download the PDF and CSV report, then open *Audit log* and show the entry with the admin and time. Finish with the dashboard totals.

Honest limitations (say them before you are asked)

- Payments are simulated, as the SRS requires for Release 1.0. A real gateway is a future release.
- The database is created with `create_all`; a migration tool (Alembic) is not set up.
- Tests ran on SQLite. PostgreSQL is supported by configuration but was not exercised.
- A guest can reopen a receipt only while the browser session holds the donation token; the API has no lookup by receipt number.
- Fund reports are visible to admins only; the SRS phrase “authorized viewer” is read as Admin.

Source of truth. The numbers in this document (56 tests, 12 concurrent confirms, 13/13 and 19/19 browser checks, 0 accessibility violations) were produced by running the code in this repository. The code excerpts are copied from the files shown above each block.